

UD4. AUTOMATIZACIÓN DE TAREAS

BLOQUES DE ÓRDENES SQL

Ya hemos visto que desde un editor SQL podemos utilizar ejecutar un conjunto de órdenes SQL separados por punto y coma.

Estos bloques de instrucciones podemos guardarlos en un archivo de texto (se recomienda con extensión .sql) para poder 'cargarlo' posteriormente y volver a ejecutar las órdenes del archivo. En mysql, cuando hacemos una copia de seguridad de una base de datos, se crea un archivo .sql con todas las órdenes SQL necesarias para crear y añadir datos a las tablas.

VARIABLES DE USUARIO

Dentro de estos bloques de instrucciones podemos hacer uso de variables de usuario para guardar información de una consulta o utilizarlas dentro de una instrucción SQL, como un UPDATE, DELETE,...

Los nombres de variables deben cumplir las siguientes normas:

- Tienen una longitud máxima de 64 caracteres.
- No distinguen mayúsculas y minúsculas.
- Ceben comenzar su nombre con el símmbolo @. Por ejemplo: @var.
- Pueden incluir cualquier carácter alfanumérico.
- Pueden incluir los símbolos: ".", "_", "\$".
- Pueden incluir otro tipo de símbolos (incluído espacio en blanco) a condición de que se escriba el nombre de la variable entre comillas: '@my var', '@my-var', @"my-var", or '@my-var'.

Para asignar un valor a una variable de usuario se usa la orden **SET**. En un mismo SET se pueden asignar valores a varias variables. No tenemos que indicar el tipo de dato que se va a guardar, así que podríamos poner:

```
SET @var='valor';                ó                SET @var=1;
```

Un ejemplo de uso de variables sería:

```
SET @codigoCliente=10;

SELECT COUNT(*)
FROM pedidos
WHERE cli_codigo = @codigoCliente;
```

También podemos guardar el resultado de una consulta en una variable:

```
SELECT COUNT(*) INTO @pedidos
FROM pedidos
WHERE cli_codigo = 10;

SELECT @pedidos;
```

Podemos guardar uno o varios valores, teniendo en cuenta que cada variable se corresponde en el mismo orden con el select asociado. El número de columnas obtenidas en el select se debe de corresponder con el número de variables. Así, si el select tiene 4 columnas necesitamos 4 variables en la cláusula INTO. Por ejemplo:

```
SELECT MAX(SALARIO), MIN(SALARIO) INTO @max, @min
FROM EMPLEADOS;
```

```
SELECT @max AS 'Salario Máximo', @min as 'Salario Mínimo';
```

Es importante tener en cuenta que si estamos usando `SELECT ...INTO` debemos asegurarnos de que el select sólo devuelva una fila, ya que en caso de devolver más de uno no podría guardar los datos en el INTO y daría error.

Podemos generar 'nuevas' variables en base a valores de otras variables, por medio de expresiones.

Por ejemplo, si tengo dos variables numéricas voy a poder sumarlas, multiplicarlas, dividir las,...generando un valor nuevo que podría guardar en otra variable. En el caso de las cadenas de caracteres, debemos hacer uso de funciones. Por ejemplo:

```
SET @codigoProvincia=36, @provincia='Pontevedra';  
SET @cad= CONCAT(@codigoProvincia, ' - ', @provincia);  
SELECT @cad;
```

PROCEDIMIENTOS ALMACENADOS

Un procedimiento almacenado es un conjunto de sentencias que pueden ser almacenadas en el servidor bajo un nombre único, y que las aplicaciones cliente pueden ejecutar indicando el nombre del procedimiento y pasándole, opcionalmente, los parámetros necesarios.

El conjunto de instrucciones almacenado en el servidor ya se encuentra enlazado y optimizado para su ejecución, lo que supone una mejora en el rendimiento y una disminución del tráfico en la red.

Algunas de las utilidades de los procedimientos almacenados:

- ➔ Cuando muchas aplicaciones, escritas en distintos lenguajes y plataformas, necesitan realizar un conjunto de operaciones susceptibles de almacenar en el servidor.
- ➔ Cuando la seguridad es una prioridad. Los bancos, por ejemplo, utilizan los procedimientos almacenados para todas las operaciones estándar, y no lo dejan en manos de los programadores de las aplicaciones. De esta forma se aseguran de que las operaciones son hechas y registradas correctamente. En una situación de este tipo, las aplicaciones cliente no acceden directamente a las tablas, sino que lo hacen a través de los procedimientos almacenados, que son los encargados de manejar los datos en la base de datos.

Los procedimientos almacenados se crean dentro de una base de datos concreta utilizando la sentencia `CREATE PROCEDURE`. Si no indicamos la base de datos, el procedimiento se creará en la base de datos activa. Al crear un procedimiento, como mínimo, debemos indicar un nombre para el procedimiento y una secuencia de órdenes SQL entre las palabras clave `BEGIN` y `END`:

```
CREATE PROCEDURE nombre_procedimiento ( [parámetro [, ...]])  
[característica ...] cuerpo_procedimiento
```

donde:

parametro: [IN | OUT | INOUT] nombre_parámetro tipo

tipo: Cualquier tipo de dato válido en MySQL

característica: LANGUAGE SQL | SQL SECURITY { DEFINER | INVOKER } | COMMENT 'string'

cuerpo_procedimiento: Secuencia de instrucciones SQL

- Los paréntesis que contiene la lista de parámetros son obligatorios, y si no se necesita ninguno no se escribirá nada en ellos. El procedimiento almacenado puede devolver resultados mediante los parámetros OUT e INOUT. Si no se indica el tipo del parámetro, se considera que es de entrada (IN).
- **SQL_SECURITY** indica si en la ejecución del procedimiento almacenado se utilizan privilegios del creador del procedimiento (DEFINER - valor por defecto) o del usuario del procedimiento (INVOKER).
- **COMMENT** permite introducir un comentario que se verá al ejecutar la sentencia SHOW CREATE PROCEDURE.

Para ver la información de los procedimientos almacenados debemos ejecutar la orden SHOW PROCEDURE STATUS. Por ejemplo, si queremos ver los procedimientos almacenados de la base de datos 'EMPRESA' sería:

```
SHOW PROCEDURE STATUS WHERE DB='EMPRESA';
```

Para ver el código de un procedimiento almacenado se usa la orden SHOW CREATE PROCEDURE nombre

```
SHOW CREATE PROCEDURE nombre_proc;
```

Para ejecutar un procedimiento almacenado ya creado en el servidor se utiliza la sentencia CALL, con la siguiente sintaxis:

```
CALL nombre_procedimiento( [ parámetro [, ...] ] )
```

DELIMITER

mysql ejecuta la sentencia cuando encuentra un ;

Lo habitual para separar una instrucción de otra, es finalizarlas con un ";" pero un procedimiento almacenado, función o Trigger, están formados por varias instrucciones, por lo tanto, antes de comenzar su definición, debemos especificar qué carácter delimita (o separa) un procedimiento o Trigger del siguiente procedimiento o Trigger. Por ello, para poder usar el ; en nuestros procedimientos y funciones, debemos cambiar el delimitador.

DELIMITER permite cambiar el delimitador. Entre los más comunes están: // o \$\$. Por ejemplo:

DELIMITER //		DELIMITER \$\$
...	ó bien	...
//		\$\$

EJEMPLO DE PROCEDIMIENTO

```
DELIMITER //
```

```
CREATE PROCEDURE sp_contar_pedidos ( pcli_codigo INT, OUT num_pedidos INT)
BEGIN
    SELECT COUNT(*) INTO num_pedidos
    FROM pedidos
    WHERE cli_codigo = pcli_codigo;
END
//
DELIMITER ;
```

Para ver los pedidos que tiene el cliente de código 1 llamaríamos al procedimiento ejecutando la siguiente instrucción:

```
CALL sp_contar_pedidos(1, @var ) ;
SELECT @var;
+----+
|@var|
+----+
| 21 |
+----+
```

MODIFICAR PROCEDIMIENTOS

En MYSQL no se permite modificar los parámetros y el cuerpo de un procedimiento. Para hacerlo habría que eliminar y volver a crear el procedimiento. Pero sí que se pueden modificar algunas características utilizando la instrucción ALTER PROCEDURE, como la cláusula COMMENT, que sirve para tener una descripción del procedimiento.

```
ALTER PROCEDURE sp_contar_pedidos
COMMENT 'Cuenta el número de pedidos del cliente que se le pasa como parámetro';
```

ELIMINAR PROCEDIMIENTOS

Para eliminar un procedimiento en MYSQL se usa la instrucción DROP PROCEDURE. Se puede incluir una cláusula 'IF EXISTS' para indicar que solamente borre el procedimiento en caso de que exista. Esto es muy útil cuando tenemos un bloque de sentencias, ya que si no ponemos el IF, si el procedimiento no existe daría un error y se dejarían de ejecutar las sentencias.

```
DROP PROCEDURE IF EXISTS sp_procedimiento1;
```

VARIABLES EN PROCEDIMIENTOS

Podemos tener variables dentro del procedimiento para guardar datos temporalmente y utilizarlos en otras instrucciones dentro del mismo. Existen algunas diferencias entre estas variables y las usadas en los bloques de código SQL:

- El nombre de las variables no empieza por @.
- Si no se inicializan, el valor de dichas variables por defecto es null.
- Se debe declarar un tipo asociado a la variable. Esto no impide que se pueda asignar un dato a la variables que no se corresponde con el tipo, en cuyo caso se hace la conversión (siempre que pueda, como por ejemplo, asignar a una variable de cadena un valor numérico, pero no podría a la inversa).

Para asignar un valor a una variable se usa el comando SET de la forma:

```
SET variable := valor;           ó bien           SET variable = valor;
```

Por ejemplo:

```
DELIMITER //
```

```
CREATE PROCEDURE sp_total_pedidos ()  
  COMMENT 'Indica el total de pedidos'  
  BEGIN  
    DECLARE numPedidos int default 0;  
  
    SELECT COUNT(*) INTO numPedidos  
    FROM pedidos;  
  
    SELECT numPedidos;  
  END  
//  
DELIMITER ;
```

PARÁMETROS

Los procedimientos almacenados pueden recibir y devolver parámetros. Los parámetros son variables cuyo valor se utiliza dentro del procedimiento almacenado.

Existen parámetros de entrada y de salida.

En los parámetros **de entrada**, cualquier modificación que hagamos sobre el valor del parámetro dentro del procedimiento almacenado no tiene efecto fuera de él. Si no se indica nada los parámetros serán de entrada.

Los parámetros **de salida** son parámetros que son utilizados por el procedimiento almacenado para devolver un valor al que llama al procedimiento. Se asigna un valor a dicho parámetro dentro del procedimiento y este valor será utilizado posteriormente por quien realizó la llamada, leyendo el valor del parámetro modificado. Para indicar que un parámetro es de salida debemos añadir la palabra clave OUT.

Los parámetros que pueden llevar información al procedimiento, que pueden ser modificados dentro del procedimiento y que pueden ser utilizados por quien realizó la llamada al procedimiento se denominan **de entrada/salida**.

En el procedimiento de ejemplo de la página 4 tenemos un parámetro de entrada y otro de salida.

INSTRUCCIONES CONDICIONALES

Al igual que en otros lenguajes de programación, en MYSQL la sentencia IF-THEN-ELSE se usa para ejecutar el código cuando una condición es VERDADERA, o ejecutar un código diferente si la condición se evalúa como FALSA.

La sintaxis de la declaración IF-THEN-ELSE en MySQL es:

```
IF condicion_1 THEN  
  {...instrucciones que se ejecutan si la condición_1 es VERDADERA...}  
  
[ ELSEIF condicion_2 THEN  
  {...instrucciones que se ejecutan si la condición_1 es falsa y  
    la condición_2 es VERDADERA...} ]  
  
[ ELSE  
  {... instrucciones que se ejecutan si son FALSAS la condición_1 y  
    la condición_2 } ]  
  
END IF;
```

La sentencia CASE tiene la funcionalidad de una sentencia IF-THEN-ELSE cuando hay varias opciones a evaluar.

Hay dos posibles sintaxis:

```

CASE expression
  WHEN valor1 THEN
    {...instrucciones a ejecutar cuando la expresión es igual a valor1 ...}
  [ WHEN valor2 THEN
    {...instrucciones a ejecutar cuando la expresión es igual a valor2 ...} ]
  [ WHEN valueN THEN
    {...instrucciones a ejecutar cuando la expresión es igual a valorn...} ]
  [ ELSE
    {...instrucciones a ejecutar cuando no es ninguno de los
      valores indicados ...} ]
END CASE;

```

ó bien

```

CASE
  WHEN condicion1 THEN
    {...instrucciones que se ejecutan si la condición1 es VERDADERA...}
  [ WHEN condicion2 THEN
    {...instrucciones que se ejecutan si la condición2 es VERDADERA...} ]
  [ WHEN condicionN THEN
    {...instrucciones que se ejecutan si la condición_N es VERDADERA...} ]
  [ ELSE
    {...instrucciones que se ejecutan si todas las condiciones son FALSAS...} ]
END CASE;

```

EJEMPLO:

Ejemplo de creación y uso de un procedimiento almacenado para llevar un registro de los intentos fallidos de inicio de sesión:

```

DROP PROCEDURE IF EXISTS sp_error_login1;

delimiter //

CREATE PROCEDURE sp_error_login (pUsuario CHAR(16), pPalabraPaso CHAR(16),
                                OUT mensaje VARCHAR(50))
BEGIN
  DECLARE errorAnterior DATETIME;
  DECLARE errorActual DATETIME;
  DECLARE intentos INT;
  DECLARE difSegundos INT;

  SET errorActual = NOW();

  SELECT count(*) INTO intentos FROM LogError WHERE UsuarioID = pUsuario;

  IF intentos < 3 THEN
    SET mensaje = "Revise los datos de usuario y contraseña";
  ELSE
    SET mensaje = "Se bloqueará la cuenta de usuario";

    UPDATE Clientes
    SET PalabraPaso = "asdfg1(23-78SD"
    WHERE ClienteID = pUsuario;
  END IF;

  IF intentos > 0 then

```

```

        SELECT Fecha INTO errorAnterior
        FROM LogError
        WHERE UsuarioID = pUsuario
        ORDER BY Fecha DESC Limit 1;

        SET difSegundos= TIME_TO_SEC(TIMEDIFF(errorActual,errorAnterior));

        IF difSegundos > 3 THEN
            INSERT INTO LogError (UsuarioID, PalabraPaso, Fecha)
            VALUES (pUsuario, pPalabraPaso, errorActual);
        END IF;
    ELSE
        INSERT INTO LogError (UsuarioID, PalabraPaso, Fecha)
        VALUES (pUsuario, pPalabraPaso , errorActual);
    END IF;
END
//
delimiter ;

call sp_error_login(1,"111",@met);
SELECT @met;
SELECT * FROM LogError WHERE UsuarioID = 1 ORDER BY Fecha DESC;

```

INSTRUCCIONES REPETITIVAS

Existen varias opciones para crear un bucle en el que un conjunto de instrucciones se repiten hasta o mientras se cumpla una determinada condición.

La instrucción **WHILE** se usa cuando no está seguro de cuántas veces ejecutará el cuerpo del bucle y es posible que el cuerpo del bucle no se ejecute ni una sola vez.

La sintaxis de la sentencia WHILE en MySQL es:

```

[etiqueta:] WHILE condicion DO
    {...instrucciones...}
END WHILE [etiqueta];

```

Dado que la condición de WHILE se evalúa antes de entrar al bucle, es posible que el cuerpo del bucle no se ejecute ni una sola vez.

la instrucción **REPEAT** se usa cuando no se sabe cuántas veces desea que se ejecute el cuerpo del bucle. La sintaxis de la sentencia REPEAT en MySQL es:

```

[etiqueta:] REPEAT
    {...instrucciones...}
UNTIL condicion
END REPEAT [etiqueta];

```

CURSORES

Los cursores nos permiten almacenar una conjunto de filas de una tabla en una estructura de datos que podemos ir recorriendo de forma secuencial.

Los cursores son de sólo lectura: No permiten actualizar los datos. Además, sólo pueden ser recorridos en una dirección y no podemos saltarnos filas.

Cuando declaramos un cursor dentro de un procedimiento almacenado debe aparecer antes de las declaraciones de los manejadores de errores (HANDLER) y después de la declaración de variables locales.

El proceso para utilizar un cursor suele ser siempre el mismo en todos los gestores de bases de datos que lo soportan:

1. Declarar el cursor: Donde se incluye la instrucción SELECT que genera un conjunto de resultados.

```
DECLARE nombreCursor CURSOR FOR SENTENCIA_SQL_SELECT;
```

2. Abrir el cursor:

```
OPEN nombreCursor;
```

3. Recorrer el cursor hasta que no haya más filas.

Al recorrer el cursor vamos guardando los datos (indicados por las columnas de la sentencia SELECT) en variables .

La forma que tiene Mysql de saber cuando ha llegado al final del cursor es 'capturando' una excepción. Esto lo hace declarando una variable (en el ejemplo done) que valdrá TRUE cuando el cursor llegue al final:

```
DECLARE done INT DEFAULT FALSE;  
DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
```

Para recorrer el array tendremos que utilizar la orden FETCH, que guarda en variables las columnas indicadas en el SELECT del cursor.

4. Cerrar el cursor

```
CLOSE nombreCursor;
```

GESTIÓN DE ERRORES

Cuando trabajamos con procedimientos almacenados en MySQL tenemos que tener en cuenta que durante la ejecución de nuestra aplicación se pueden producir errores.

Un ejemplo podría ser cuando intentamos dar de alta (INSERT) una fila con una clave primaria duplicada. Al intentar hacer esto, el gestor 'rompe' la ejecución del procedimiento almacenado en la línea del INSERT e informa del problema.

Otro ejemplo sería, si estamos trabajando con el motor de base de datos InnoDB y definimos claves foráneas, podrán producirse errores de integridad referencial si intentamos hacer un INSERT en un campo que clave ajena y el valor que intentamos introducir no existe en la tabla que referencia.

Para llevar el control de estos errores podemos definir **handlers** en nuestro procedimientos almacenados .

Cada error en MySQL genera un código de error, el cual tendremos que saber para poder manejarlo en nuestro handler.

Así, en nuestro procedimiento, debemos 'declarar' primero cuales son los errores que se van a capturar y cuales serán las instrucciones que se deben ejecutar en caso de que se produzca. Esto hará que el procedimiento almacenado continúe la ejecución con la siguiente línea a la que provocó el error.

La sintaxis es:

```
DECLARE {exit | continue } HANDLER FOR valor_condición [ , ... ]  
sentencia
```

```
Valor_condición = SQLSTATE[VALUE] valor_sqlstate | nombre_condición | código_error_mysql
```

EJEMPLO:

Ejemplo de creación de un procedimiento almacenado para contar las compras realizadas por nuestros clientes, y en función de los resultados modificar un campo de la tabla de clientes que debería almacenar el número de compras que ha realizado (clt_compras):


```
DROP PROCEDURE IF EXISTS ventas_todos;
DELIMITER //
CREATE PROCEDURE ventas_todos()
BEGIN
    DECLARE cliente, contar INTEGER;
    DECLARE final INTEGER DEFAULT 0;

    /* Declaración de un cursor para recorrer los resultados de la consulta que
    devuelve los números de todos los clientes que hicieron compras, y el número
    de compras que hizo cada uno */
    DECLARE c1 CURSOR FOR
        SELECT ven_clt, COUNT(*) FROM ventas GROUP BY ven_clt;

    /* Declaración de un manipulador para la condición de error de final de
    lectura de resultados */
    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET final=1;
    OPEN c1;
    REPEAT
        /* Avanzamos una línea de resultados y pasamos las columnas de la selección a
        las variables */
        FETCH c1 INTO cliente, contar;
        IF NOT final THEN          /* si no se llegó al final de los resultados
                                   modificamos los datos del cliente */
            UPDATE clientes SET clt_compras = contar
            WHERE clt_num = cliente;
        END IF;
    UNTIL final END repeat;
    CLOSE c1;
END
//
DELIMITER ;
```